

Lecture 2

Eleana Cabello

Let's get on the same page

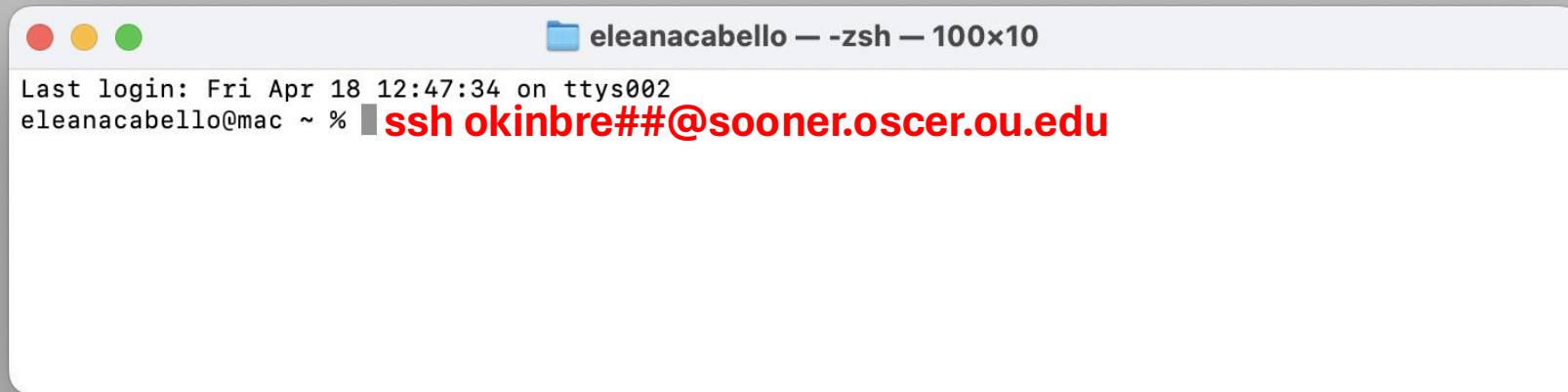
- Windows systems – Log onto OSCER using MobaXterm
- Mac systems - Find the Terminal application



- **Terminal** - A text-based user interface that allows users to interact with a computer by typing commands
- **Directory** - A folder location in your computer
- **Working Directory** - The current directory a user is in
- **Path** - Directions to a folder or file

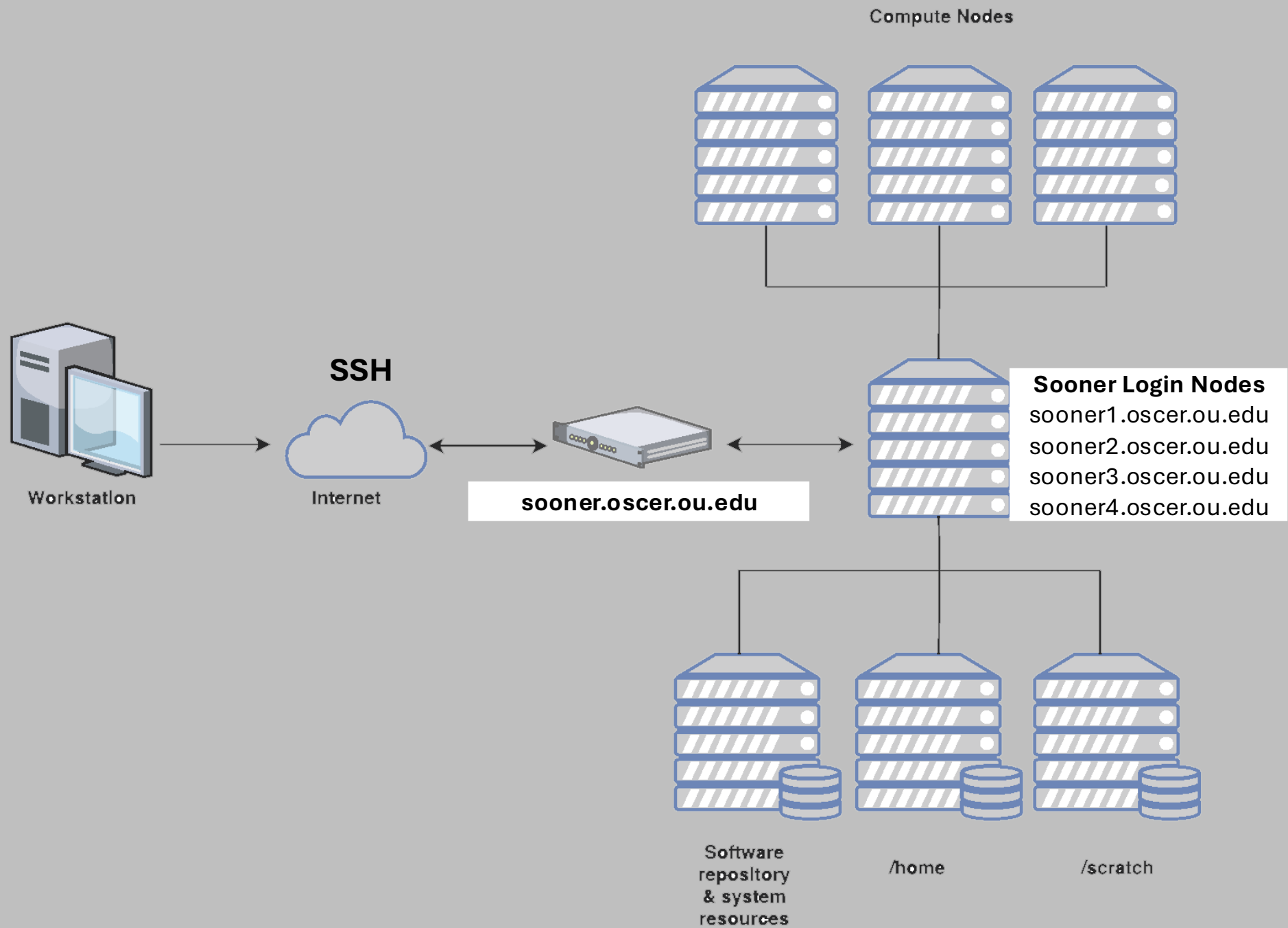
Let's log into your guest accounts

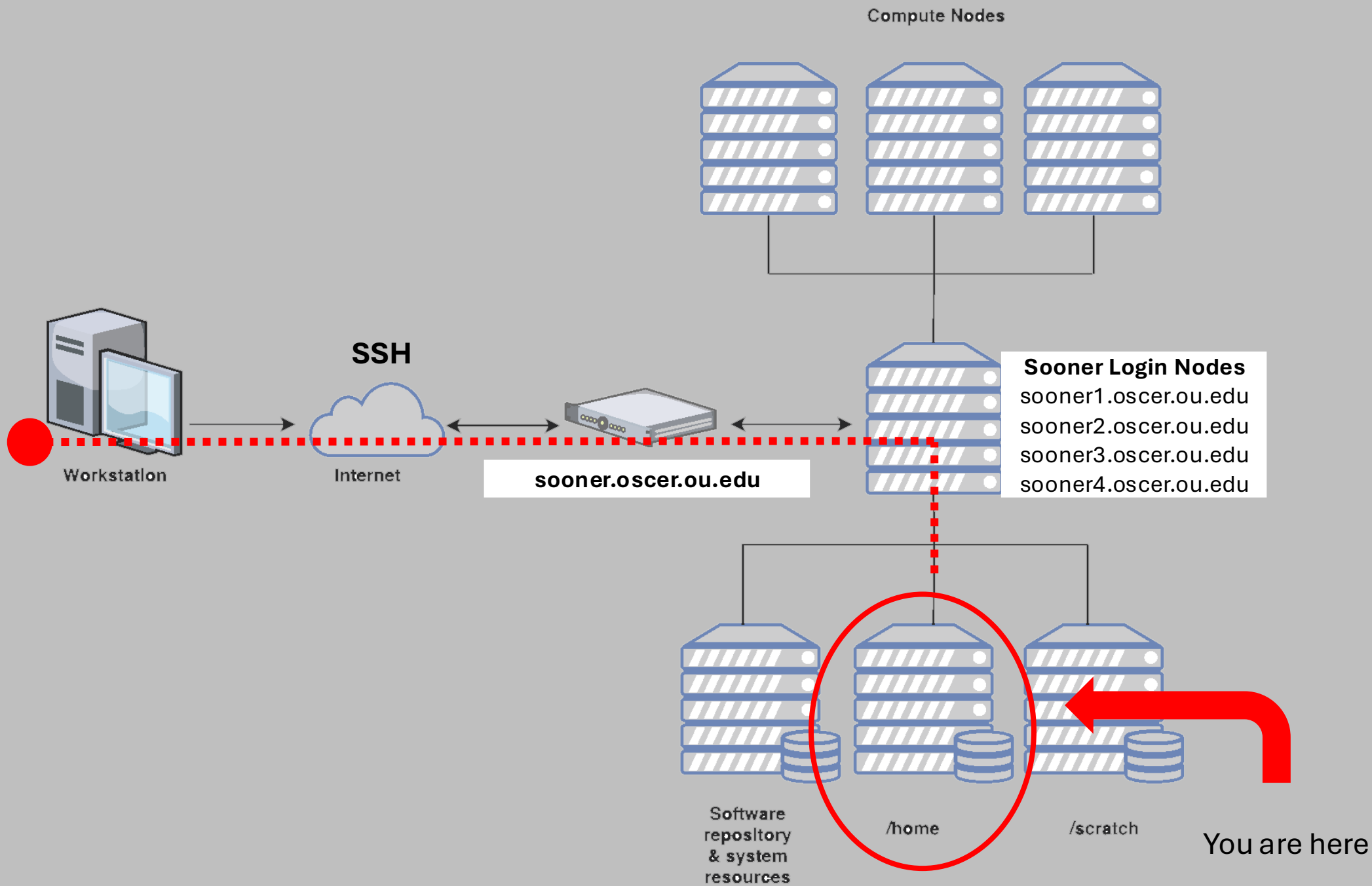
- Gather you log in information, username and password.

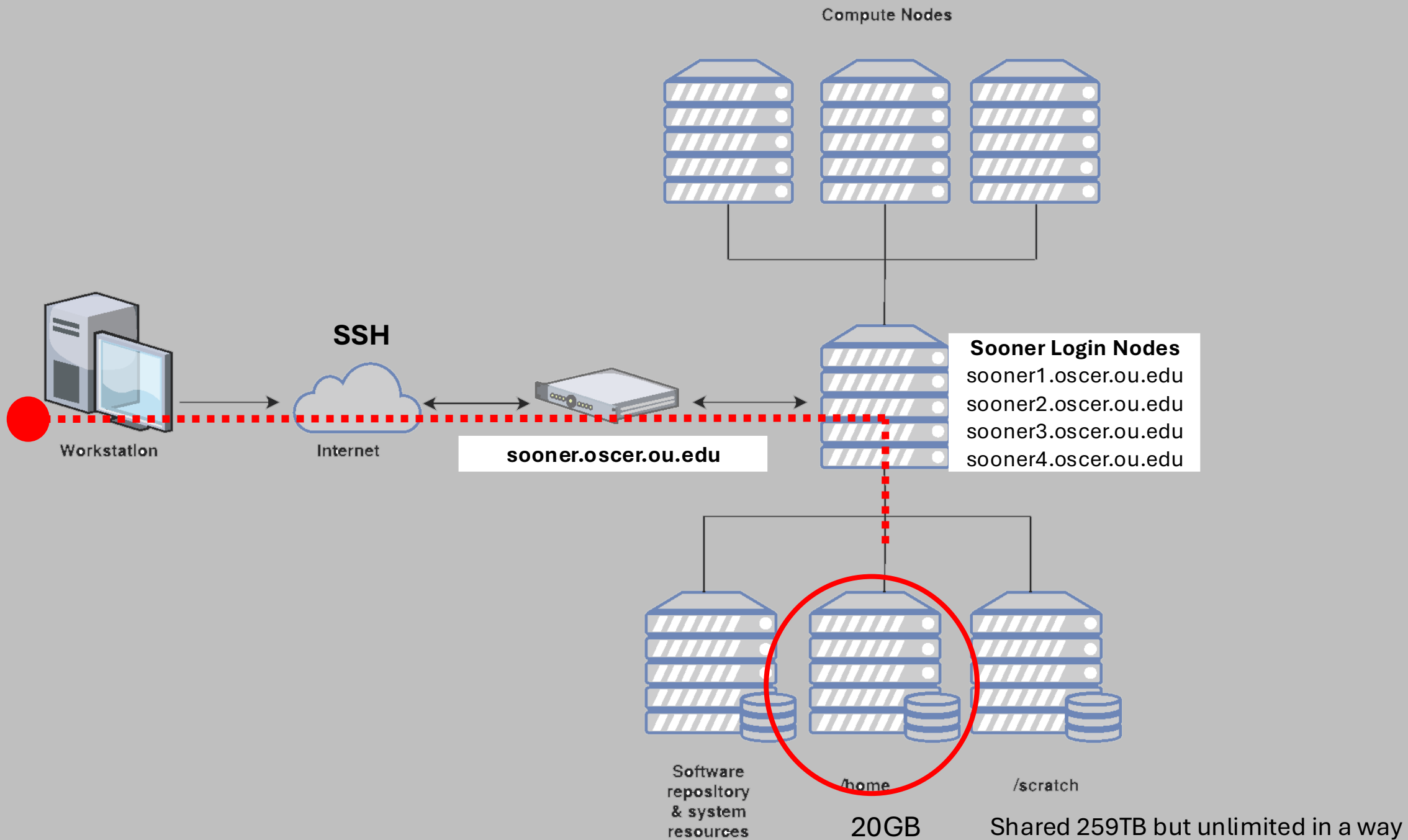
A screenshot of a macOS terminal window. The title bar at the top shows three colored window control buttons (red, yellow, green) on the left, followed by a blue folder icon and the text "eleanacabello — -zsh — 100x10". The terminal content shows the output of a last login: "Last login: Fri Apr 18 12:47:34 on ttys002". Below this, the prompt "eleanacabello@mac ~ %" is followed by a red cursor block and the command "ssh okinbre##@sooner.oscer.ou.edu" which is also displayed in red text.

```
eleanacabello — -zsh — 100x10
Last login: Fri Apr 18 12:47:34 on ttys002
eleanacabello@mac ~ % ssh okinbre##@sooner.oscer.ou.edu
```

- Type the following command to access you OSCER account: **ssh okinbre##@sooner.oscer.ou.edu**
- It will ask for your password, start typing it. It will not appear on your screen do not be alarmed this is normal. Just make sure to enter it correctly then hit enter.







BASH Programming

From where you are type: **cp -r /scratch/elec/lecture_2 /scratch/okinbre##**

Now type: **cd /scratch/okinbre##**

Navigating Directories

<code>pwd</code>	<code># Print working directory</code>
<code>ls</code>	<code># List contents in a directory</code>
<code>ls -lh</code>	<code># List contents in a directory in a list format and human read able sizes</code>
<code>tree</code>	<code># List directory and file tree</code>
<code>cd DIR</code>	<code># Change working directory to the entered directory</code>
<code>cd ..</code>	<code># Change working directory to parent directory of current directory</code>
<code>cd ~</code>	<code># Change working directory to home</code>

Creating, Moving, and Deleting Directories

```
mkdir DIR      # Create directory
```

```
rmdir DIR      # Remove Directory
```

```
rm -r DIR      # Remove Directory
```

```
mv ONE_DIR TWO_DIR  # Move directory to new directory
```

Deleting, Copying, and Moving Files

```
rm FILE.txt           # Remove file
cp FILE.txt FILE.txt  # Copy file to new directory
mv FILE.txt FILE.txt  # Move file to new directory
```

There is no back button once things are deleted or overwritten it is gone

Command Line Text Editor

```
nano file_nano_test.txt      # Opens file in line for editing
```

Navigate with up, left, right, and down keys

Type as usual

Exit using CTRL+ X

```
Save modified buffer?
Y Yes
N No      ^C Cancel
```

The above message will appear because you made changes, click Y on your keyboard

```
File Name to Write: test.sh
^G Help      M-D DOS Format      M-A Append      M-B Backup File
^C Cancel    M-M Mac Format      M-P Prepend     ^T Browse
```

It will then ask about the filename you want to save it to, to accept click the ENTER key on your keyboard

```
#!/bin/bash
#SBATCH --partition=normal
#SBATCH --ntasks=1
#SBATCH --mem=1G
#SBATCH --job-name=jobname
#SBATCH --output=jobname_%J_%A_%a_stdout.txt
#SBATCH --error=jobname_%J_%A_%a_stderr.txt
#SBATCH --time=0:10:00
#SBATCH --mail-user=EMAIL@ou.edu
#SBATCH --mail-type=ALL
#SBATCH --chdir=/scratch/USER/lecture_2
```

#SBATCH Directives

Where to send your job

Number of tasks

Memory needed

Name of job

Output file name

Error file name

Time limit

Email to send updates to, **PLEASE REMEMBER TO PUT YOURS**

All notifications

Working directory of the job

```
#=====
# BASH Strict mode (i.e. "fail fast" to reduce hard-to-find bugs)
set -e          # EXIT the script if any command returns non-zero exit status.
set -E          # Make ERR trapping work inside functions too.
set -u          # Variables must be pre-defined before using them.
set -o pipefail # If a pipe fails, returns the error code for the failed pipe
                # even if it isn't the last command in a series of pipes.
#=====
```

Quick Exercise

Change into your scratch: **cd /scratch/okinbre##**

Run this: **pwd**

**Make sure you are in
/scratch/user/lecture_2!**

Then type: **nano first_exercise.sh**

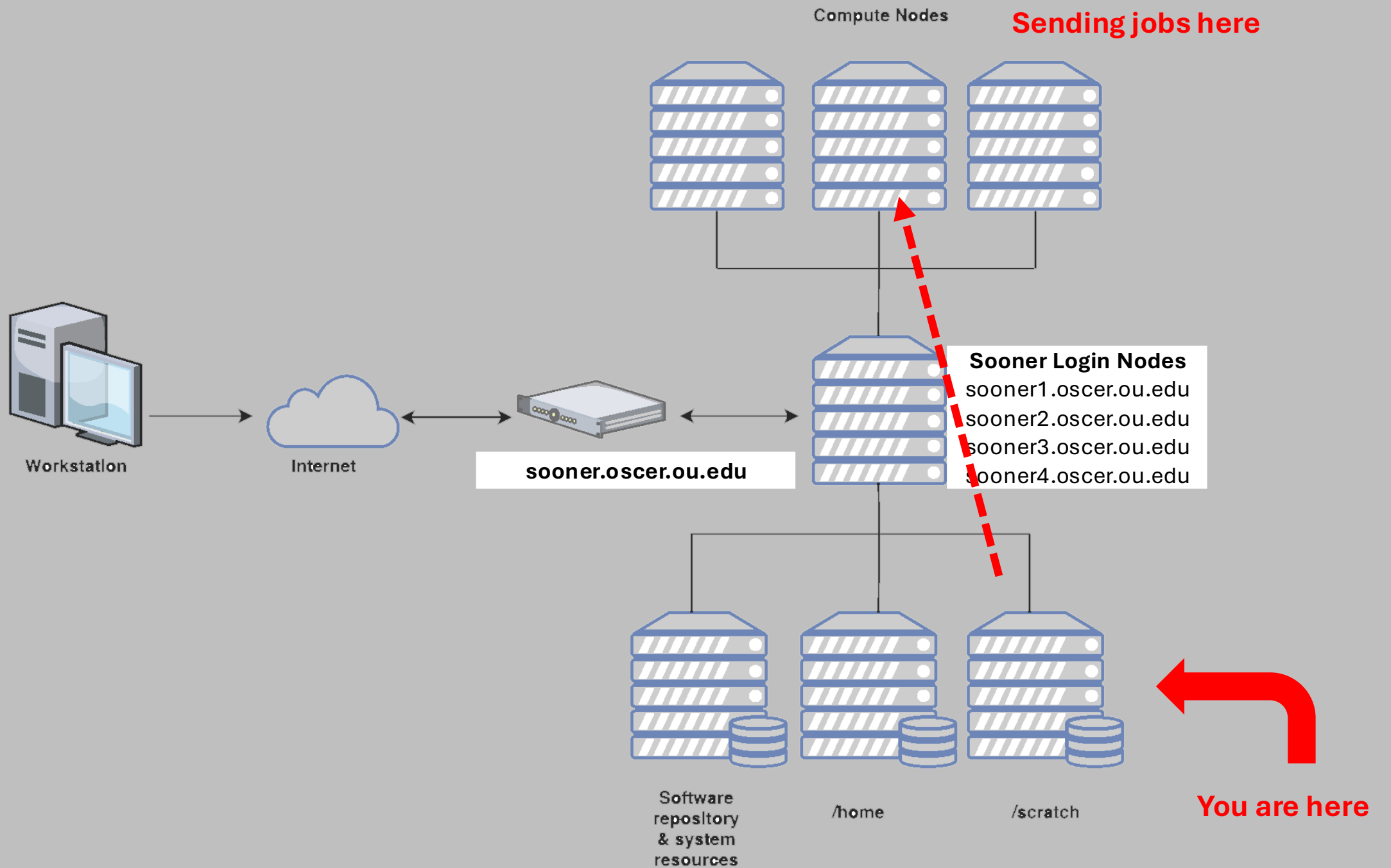
```
#!/bin/bash
#SBATCH --partition=normal
#SBATCH --ntasks=1
#SBATCH --mem=1G
#SBATCH --job-name=jobname
#SBATCH --output=jobname_%J_%A_%a_stdout.txt
#SBATCH --error=jobname_%J_%A_%a_stderr.txt
#SBATCH --time=0:10:00
#SBATCH --mail-user=EMAIL@ou.edu
#SBATCH --mail-type=ALL
#SBATCH --chdir=/scratch/USER/lecture_2
```

 **CHANGE THIS TO YOUR EMAIL as in firstname-lastname@ou.edu**

```
#=====
# BASH Strict mode (i.e. "fail fast" to reduce hard-to-find bugs)
set -e      # EXIT the script if any command returns non-zero exit status.
set -E      # Make ERR trapping work inside functions too.
set -u      # Variables must be pre-defined before using them.
set -o pipefail # If a pipe fails, returns the error code for the failed pipe
              # even if it isn't the last command in a series of pipes.
#=====
```

```
echo "Hello World!" > helloWorld.txt
```

 **> redirects output into a file**



Useful Tips and Questions

Before you leave today

- ☐ Have R and RStudio installed